IT112 – WEB SYSTEMS TECHNOLOGIES
BSIT-2B 2nd Semester

Campus Service Hub:
A Student Guide and Walkthrough in Bicol University Polangui

Group Name:
Code: <breakers>

Members:
Carilla, Grace Ann S.
Lamud, Ella Mae C.
Olarcos, Joshua L.
Requio, Rein Ashley S.
Rubio, Ma. Miscy M.
Sentillas, Tiffany C.

Professor
Guillermo V. Red, Jr

May 2026

Overview of Data Retrieval and Display

Phase 5 represents a significant milestone in the development of the Campus Service Hub, as it transitions the system from a data-insertion model into a fully interactive application capable of retrieving and dynamically rendering real data on the frontend. Having established functional POST routes and form submission logic in the preceding phase, the focus in this phase shifted toward building the GET-driven layer of the application — enabling the dashboard and listing pages to pull live data from the MongoDB database and present it to users through dynamically generated HTML elements.

The group developed display pages for the primary entities in the system, implementing JavaScript logic that fetches JSON responses from the backend API and renders the data as structured cards and table rows using DOM manipulation. Careful attention was given to ensuring that the displayed data accurately reflected the schema defined in the ERD, and that the interface remained responsive and readable across different screen sizes through the continued use of Bootstrap 5.

Task 1: Verification of Backend GET Endpoints

Prior to building the display interfaces, the group re-verified that all necessary GET endpoints on the backend were returning data in the correct format. Each route was tested individually using Postman and Thunder Client, and the JSON responses were inspected to confirm that the field names, data types, and document structures matched the schema defined in the User and product models. The following routes were confirmed operational:

- GET /api/users – Returns an array of all registered user documents from the users collection.
- GET /api/products – Returns an array of all product entries from the products collection.

The backend developer also reviewed the controller functions responsible for handling these routes to ensure that they were querying the database efficiently and returning clean, complete JSON payloads. No schema mismatches or missing fields were found, confirming that the backend was ready to serve data to the frontend display layer.

Task 2: Dashboard and Display Page Development

Two primary display pages were developed during this phase to serve as the data-facing interfaces of the Campus Service Hub. The dashboard.html page was designed as the central hub accessible to authenticated users, providing an at-a-glance view of relevant system data. A dedicated admin-users.html page was also created to allow administrators to view and oversee all registered user accounts within the system.

Each page was structured with clearly defined container elements bearing specific id attributes, which served as the target mounting points for the dynamically generated HTML content. This approach kept the static HTML markup minimal and delegated all content

rendering responsibilities to the corresponding JavaScript files, resulting in a clean separation between structure and behavior.

Task 3: JavaScript Fetch and DOM Rendering Logic

The data retrieval and rendering logic was written in modular JavaScript files linked to their respective display pages. On page load, an asynchronous fetch() call is dispatched to the appropriate GET endpoint, and the returned JSON array is iterated using the forEach() method. For each data object in the response, a new HTML element is dynamically created, populated with the relevant fields, assigned Bootstrap card classes for consistent styling, and appended to the designated container in the DOM.

The rendering functions were written to be reusable and easily adaptable for other entities in the system. By parameterizing the fetch URL and the fields to display, the same core logic can be repurposed for products, services, or any other collection without significant restructuring. Error handling was also incorporated to account for failed fetch calls or empty datasets, ensuring that the interface communicates clearly to the user when data is unavailable rather than silently rendering a blank page.

The displayed output was cross-referenced against the actual documents in the MongoDB Atlas collection to verify that the data rendered on the frontend was accurate and complete, with no field omissions or formatting inconsistencies.

Task 4: Search and Pagination

To improve usability on the data listing pages, a client-side search filter was implemented using vanilla JavaScript. A search input field was added to the admin-users.html page, and an event listener was attached to it that responds to every keystroke. As the user types, the rendered list is filtered in real time by comparing the input value against the name field of each user object, and only the matching entries are displayed in the container.

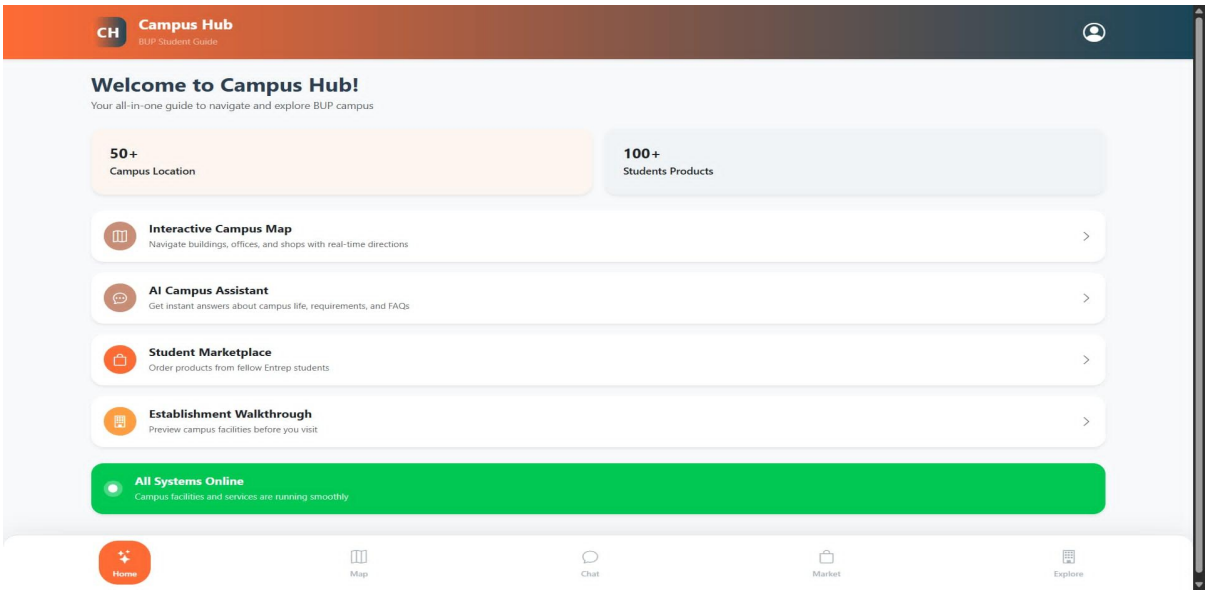
This approach avoids additional API calls for each search query, instead operating on the dataset already loaded into memory from the initial fetch. While basic in its current implementation, the filter provides a meaningful improvement to the user experience on pages where the number of records may grow substantially over time. Future iterations of the system can extend this functionality to include multi-field filtering or backend-side search for more complex queries.

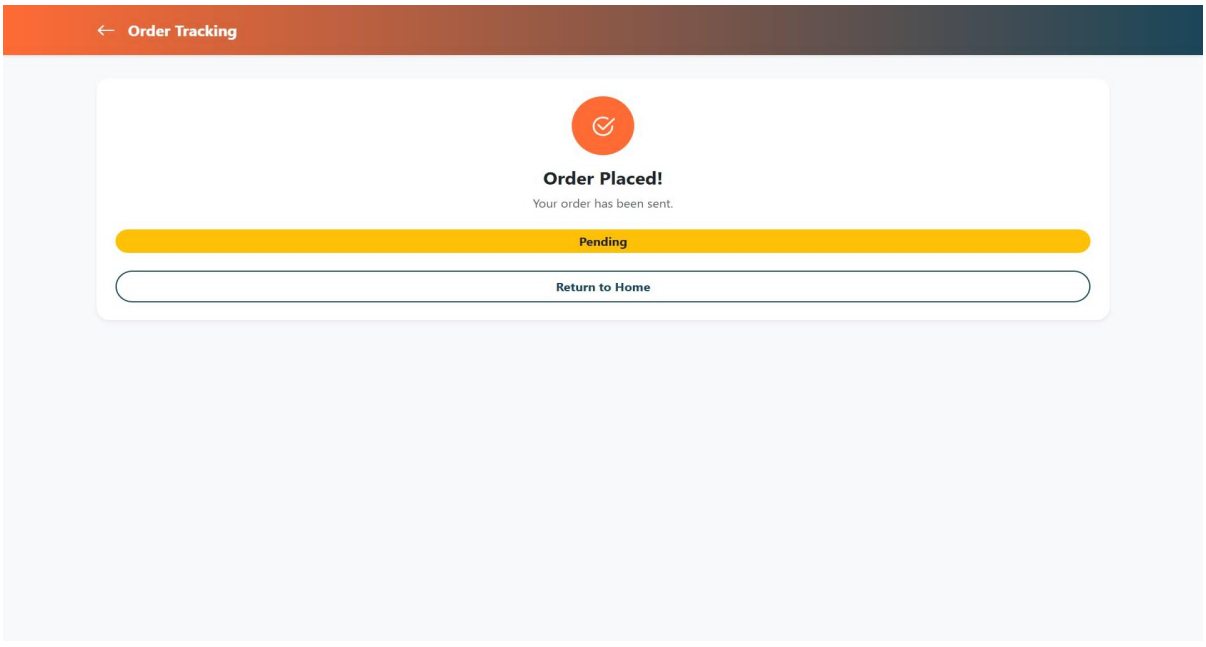
Task 5: GitHub Workflow and Repository Updates

All files produced or modified during this phase were committed to the GitHub repository following the established folder conventions. Updated JavaScript files and HTML display pages were pushed under the /frontend directory, while any backend controller adjustments were committed under /backend. The GitHub Manager tagged the latest push with the phase-5 label to facilitate easy identification of the phase boundary within the project's commit history.

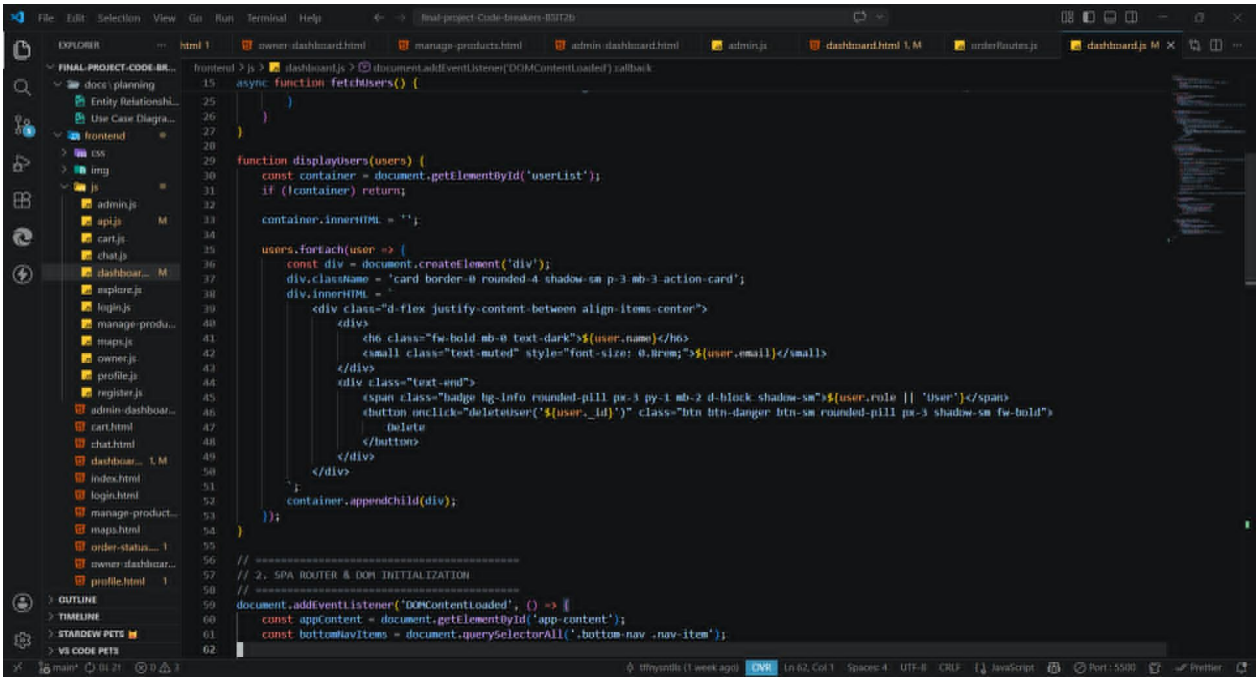
The README.md file was also updated to reflect the new functionality introduced in this phase, including a brief description of the display pages, the GET endpoints they consume, and instructions for running the full stack locally. Commit messages remained specific and informative throughout, such as "feat: Added dynamic user listing to admin-users.html" and "fix: Handled empty response in dashboard fetch," maintaining the transparency and traceability that has been upheld across all phases of the project.

Frontend Pages

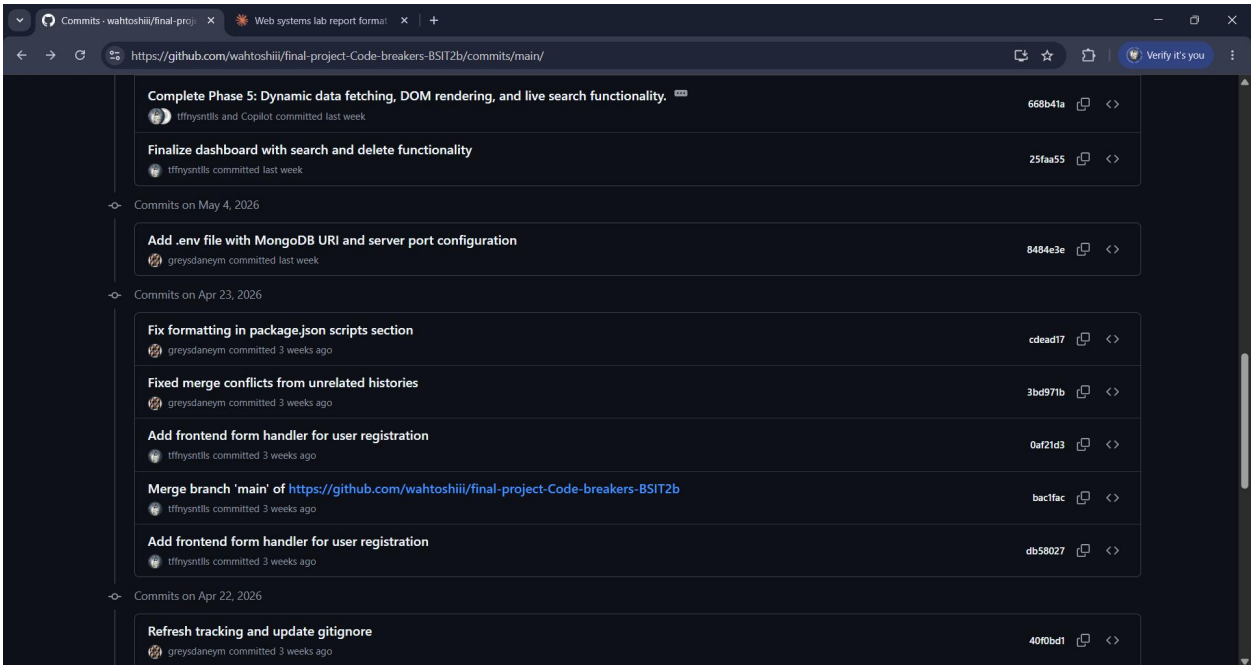




Code Snippets



GitHubCommits



Contribution of Each Member

Carilla, Grace Ann S. – As the Backend Developer, I reviewed and enhanced the existing GET route controllers to ensure they returned clean and complete JSON responses suitable for frontend rendering. I also assisted in debugging a fetch() error that occurred when the backend returned an empty array, working with the frontend developer to implement a graceful fallback message for cases where no data is available.

Lamud, Ella Mae C. – During this phase, I was unable to contribute hands-on as the GitHub Manager due to the unavailability of a personal laptop. In place of my usual responsibilities, I took on the task of compiling and writing the lab reports for the group as my way of contributing to the team's progress. My groupmates covered all repository responsibilities in my absence, including committing the updated files, applying the phase-5 tag, and updating the README, and I am genuinely grateful for their dedication. This experience reinforced for me the importance of being consistently available as a team member, and I am committed to doing better in the phases ahead.

Olarcos, Joshua L. – As the Project Manager, I oversaw the overall progress of this phase and ensured that the data displayed on the frontend accurately corresponded to the fields and relationships defined in our ERD. I coordinated the testing sessions between the frontend and backend developers and confirmed that the system behaved correctly for both populated and empty datasets before the final submission.

Requio, Rein Ashley S. – As the group's tester, I conducted thorough testing of both display pages by verifying that the dynamically rendered content matched the actual documents in the MongoDB Atlas collection. I also tested the search filter functionality under various input conditions and documented the results with screenshots, including the browser view, the console output, and the corresponding Postman API response.

Rubio, Ma. Miscy M. – As the Database Manager, I monitored the MongoDB Atlas collections throughout the testing phase to confirm that the data being rendered on the frontend was correctly sourced from the database. I also ensured that sample documents with complete and valid field values were present in the collections so that the display pages could be tested with realistic data rather than placeholder entries.

Sentillas, Tiffany C. – As the Frontend Developer and Documentation Officer, I built the display pages and wrote the JavaScript rendering logic for this phase, implementing the fetch() calls, DOM manipulation functions, and the client-side search filter. I also compiled this lab report, organized the required screenshots and annotated code snippets, and ensured that all documentation was complete and accurately represented the work completed during Phase 5.